软件详细设计说明书

1. 引言

1.1 编写目的

本文档为 Shadow Filter H265 系统的详细设计说明书，旨在对系统的各个功能模块、类结构、接口定义及核心算法进行微观层面的详细描述，为代码实现和后续维护提供直接依据。

1.2 适用范围

本文档适用于开发人员、测试人员及系统维护人员。

---

2. 系统类设计 (Class Design)

2.1 视频采集类（`AsyncCameraCapture`）

负责单一视频源的异步采集，确保采集过程不阻塞主线程。

- 属性:
  - `src (int/str)`: 视频设备索引或路径（如 `/dev/video0`）。
  - `cap (cv2.VideoCapture)`: OpenCV 视频捕获对象。
  - `queue (collections.deque)`: 帧缓冲区，`maxlen=1`，仅保留最新帧。
  - `running (bool)`: 线程运行控制标志。
  - `lock (threading.Lock)`: 线程安全锁。
  - `reconnect_interval (float)`: 重连间隔（默认 2.0s）。
- 方法:
  - `__init__(src, width=720, height=576, fps=25)`: 初始化采集参数。
  - `start()`: 启动采集线程。
  - `stop()`: 停止线程并释放资源。
  - `read()`: 从队列获取最新帧（非阻塞），返回 `(ret, frame)`。
  - `_update()`: 内部线程循环方法，执行 `cap.read()` 并写入队列；包含断线重连逻辑。

2.2 AI 推理类（`OptimizedMemoryFrameProcessor`）

封装 YOLOv8 模型，负责多路视频的目标检测。

- 属性:
  - `model (ultralytics.YOLO)`: 加载的 YOLO 模型实例。
  - `batch_size (int)`: 推理时的 Batch 大小（默认 4）。
  - `conf_threshold (float)`: 置信度阈值。
  - `input_buffer (np.ndarray)`: 预分配的输入 Tensor 内存。

- 方法：
  - `process_batch(frames)`：接收帧列表，执行批量推理。
    - 优化细节：使用 Numpy `view` 切片而非 `copy` 组合 Batch，减少内存带宽占用。
  - `_post_process(results)`：解析模型输出，提取 Bounding Box 和 Class ID。

2.3 核心处理类（`SyncFrameProcessor`）

系统的中枢控制器，协调采集、推理、合成与推流。

- 属性：
  - `cameras (List[AsyncCameraCapture])`：摄像头实例列表。
  - `rtsp_server (RTSPStreamServer)`：RTSP 服务器实例。
  - `temporal_brightness_tracker (float)`：当前亮度增益值。
  - `target_brightness (float)`：目标亮度均值 (85.0)。
- 方法：
  - `start_processing()`：启动所有子模块。
  - `_processing_loop()`：主循环。
    - 同步读取 4 路摄像头帧。
    - 调用 `process_batch` 获取检测结果。
    - 执行 `apply_temporal_brightness` 调整亮度。
    - 执行 `apply_denoise` 降噪。
    - 调用 `rtsp_server` 推送数据。
  - `apply_temporal_brightness(frame)`：执行时域亮度稳定算法。
  - `apply_denoise(frame)`：执行中值/高斯降噪。

2.4 流媒体服务类（`RTSPStreamServer`）

管理 FFmpeg 子进程，处理 H.264/H.265 双编码推流。

- 属性：
  - `monitor_ffmpeg_processes (dict)`：监控流 FFmpeg 进程句柄表 `{'h264': Popen, 'h265': Popen}`。
  - `filter_ffmpeg_processes (dict)`：筛选流 FFmpeg 进程句柄表。
  - `publish_user/pass`：推流鉴权凭证。
- 方法：
  - `start_monitor_stream(fps)`：启动监控流 FFmpeg 管道。
  - `start_filter_stream(fps)`：启动筛选流 FFmpeg 管道。
  - `push_frame(frame, stream_type)`：向指定管道的标准输入写入 Raw Video 数据。
  - `_format_rtsp_url(path)`：生成带鉴权信息的推流 URL。
  - `_check_and_restart()`：监控 FFmpeg 进程状态，异常退出时自动重启。

2.5 ONVIF 服务类（`OnvifHttpServer`）

实现 ONVIF Profile S/T 协议的 SOAP 接口。

- 属性:
  - `bind_port`: HTTP 监听端口。
  - `visible_profiles`: 当前服务暴露的 Profile 列表（如 `['monitor']`）。
  - `snapshot_callback`: 获取截图的回调函数。
- 方法:
  - `do_POST()`: 处理 SOAP 请求入口。
  - `_verify_auth(body)`: 解析 WS-UsernameToken 进行鉴权。
  - `_soap_get_profiles()`: 返回 XML 格式的 Profile 列表。
  - `_soap_get_stream_uri()`: 返回 RTSP 播放地址。

---

3. 核心算法设计

3.1 时域亮度稳定算法 (Temporal Brightness Stabilization)

为解决模拟摄像头自动曝光波动导致的画面闪烁，采用指数移动平均 (EMA) 算法控制亮度增益。

- 公式: $$ \text{Avg}_{\text{curr}} = \text{mean}(\text{Frame}_{\text{gray}}) $$ $$ \text{Gain}_{\text{target}} = \frac{\text{Target}}{\text{Avg}_{\text{curr}} + \epsilon} $$ $$ \text{Gain}_{\text{smoothed}} = \alpha \cdot \text{Gain}_{\text{target}} + (1 - \alpha) \cdot \text{Gain}_{\text{prev}} $$ $$ \text{Pixel}_{\text{out}} = \text{clip}(\text{Pixel}_{\text{in}} \cdot \text{Gain}_{\text{smoothed}}, 0, 255) $$
- 参数:
  - Target: 85.0 (默认)
  - Alpha: 0.01 (平滑系数)

3.2 画面合成逻辑

3.2.1 Monitor Stream (全景监控)

- 分辨率: 720x720
- 布局: 2x2 网格
- 实现: 将 4 路 720x576 的原始画面各自缩放至 360x288 (或保持比例)，拼接至目标画布。

3.2.2 Filter Stream (智能筛选)

- 分辨率: 720x576
- 逻辑:
  1. 遍历 4 路检测结果。
  2. 若某路检测到目标（置信度 > 阈值），将该路画面作为输出。
  3. 若多路同时检测到，优先选择置信度最高或序号最小的一路。
  4. 若无目标，默认输出 Channel 0 或保持上一帧画面（可配置）。

4. 接口详细定义

4.1 命令行参数 (CLI)

参数	类型	默认值	描述
`--model`	str	`./best.pt`	YOLO 模型路径
`--conf`	float	`0.7`	置信度阈值
`--batch-size`	int	`32`	推理 Batch Size
`--temporal-target`	float	`85.0`	亮度目标均值
`--no-denoise`	bool	`False`	是否禁用降噪
`--public-ip`	str	`117.72.39.181`	对外宣告 IP

4.2 环境变量

变量名	必填	默认值	作用
`RTSP_PUBLISH_PASS`	否	`password147`	推流认证密码
`RTSP_READ_PASS`	否	`password147`	拉流认证密码

---

5. 异常处理设计

1. 摄像头断连：`AsyncCameraCapture` 捕获 `cv2.read()` 异常，重置 `VideoCapture` 对象，间隔 2s 重试。
2. FFmpeg 崩溃：`RTSPStreamServer` 每次推流前检查 `poll()` 状态，若进程已结束，立即重启该路推流管道。
3. MediaMTX 启动失败：`mediamtx_manager` 尝试杀掉占用端口的旧进程，若仍失败则打印错误但不退出主程序（假设外部已启动）。